OS_project2

12912

2022090000 정인호

1. code analysis

```
void
mm_get(struct mm_struct *mm)
{
  if (mm == 0) return;
  acquire(&mm->lock);
  mm->refcount++;
  release(&mm->lock);
}
```

mm_get 구현

Lock을 걸고 mm->refcount를 1 늘려주어 참조하고 있는 thread가 늘어났다는 것을 표기해줌

```
void
mm_put(struct mm_struct *mm)
{
  if (mm == 0) return;
  acquire(&mm->lock);
  mm->refcount--;
  if (mm->refcount == 0){
    release(&mm->lock);
    if (mm->pagetable){
      uvmunmap(mm->pagetable, TRAMPOLINE, 1, 0);
      uvmfree(mm->pagetable, mm->sz);
    }
    kfree((void *)mm);
  } else release(&mm->lock);
}
```

mm_put 구현

Lock을 걸고 mm->refcount를 하나 줄여 참조하고 있는 thread가 줄었다는 것을 표기하였다. 만약에 하나를 줄여서 refcount가 0이라면 더이상 참조하고 있는 thread가 없다는 뜻이므로 uvmunmap을 이용하여 pagetable내에서 trampoline의 가상 주소 영역의 매핑을 해

제한다. Uvmfree는 해당 프로세스가 유저 공간에서 사용하던 물리 메모리 페이지들을 해제하고 이 가상 주소 매핑 정보들을 들고 있던 페이지 테이블 구조를 지운다.

kfree를 통해 더이상 참조되지 않는 struct mm_struct 구조체가 점유하고 있던 메모리 공간을 kernel에 반환한다.

```
uint64
find_free_tf_va(pagetable_t pagetable)
{
    uint64 va;
    for(int i = 0; i < NTHREAD; i++){
        va = TRAPFRAME - i * PGSIZE;
        if (!ismapped(pagetable, va)){
            return va;
        }
    }
    return 0;
}
```

find_free_tf_va를 구현

각각의 thread는 각각의 trapframe이 필요하다.

이를 위해 ismapped라는 구현된 함수를 사용하여서 사용할 수 있는 tf_va를 반환하도록 구현했다.

```
static void
freeproc(struct proc *p)
{
    if (p->mm && p->mm->pagetable && p->tf_va){
        acquire(&p->mm->lock);

        uvmunmap(p->mm->pagetable, p->tf_va, 1, 0);

        release(&p->mm->lock);
    }
```

free_proc()는 해당 thread가 종료될 때, p가 사용하던 trapframe의 연결을 끊어주어야한다.

```
inho@jeong-inhoui-MacBookPro project2-vltn617 % make qemu CPUS=1
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -n
ne,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ usertests forktest
usertests starting
test forktest: PASS
PASS ALL TESTS
```

문제가 되었던 메모리 leaking이 해결되었다!

```
if (flags & CLONE_VM) {
        // === thread path ===
        np->mm = p->mm;
        mm_get(np->mm);
        np->group_leader = p->group_leader;
        np->tgid = np->group_leader->pid;

        acquire(&np->mm->lock);
       uint64 tf_va = find_free_tf_va(np->mm->pagetable);


       if (tf_va == 0){
              release(&np->mm->lock);
            goto fail;
        }
              np->tf_va = tf_va;


        if (mappages(np->mm->pagetable, tf_va, PGSIZE, (uint64)(np->trapframe), PTE_R | PTE_W) < 0){
            np->tf_va = 0;
            release(&np->mm->lock);
            goto fail;
        }
        release(&np->mm->lock);
        *(np->trapframe) = *(p->trapframe);
        np->trapframe->sp = stack;
        np->trapframe->epc = fn;
        np->trapframe->a0 = arg;
```

```
    np->ustack = (void *)((uint64)stack - ((uint64)n_pages * PGSIZE));
  }
```

Kclone 에서 추가된 부분이다.

Clone_vm이 켜져있을 때, 주소공간을 복사하는 대신 공유하도록 처리하는 스레드 생성부이다. fork()와 달리, 스레드는 부모의 주소공간을 공유해야하므로 주소공간을 복사하는 대신, mm포인터를 그대로 복사하고 mm_get()을 통해 mm->refcount 를 1 늘려준다.

Thread id를 설정하여 하나의 process에서 thread들이 묶여있는 것을 나타낸다.

각각의 thread들은 독립된 trapframe을 가져야하므로 find_free_tf_va로 가상 주소를 획득한 후 mappages를 통해 trapframe page와 연결한다.

Trapframe 구조체 안에 있는 변수들을 조작하여 thread의 상태를 저장한다.

```
struct proc *ch;
  int have_threads, pid;
  struct proc *p = myproc();

  acquire(&wait_lock);
  for(;;){
    have_threads = 0;
    for (ch = proc; ch < &proc[NPROC]; ch++){
      if (ch == p) continue;
      acquire(&ch->lock);
      if (ch->tgid == p->tgid && ch->state != UNUSED){
        have_threads = 1;
        if (ch->state == ZOMBIE){
          pid = ch->pid;
          if (stack_addr != 0 && copyout(p->mm->pagetable, stack_addr, (char *)&ch->ustack, sizeof(ch-
>ustack)) < 0){
            release(&ch->lock);
            release(&wait_lock);
            return -1;
          }
          freeproc(ch);
          release(&ch->lock);
          release(&wait_lock);
```

```
                return pid;
            }
        }
        release(&ch->lock);
    }
    if (!have_threads || killed(p)){
        release(&wait_lock);
        return -1;
    }
    sleep(p->group_leader, &wait_lock);
}
```

kjoin 구현

kjoin()의 역할은 같은 thread group 내에서 종료된 thread가 생길 때까지 기다린 후, 해당 thread의 자원을 회수하는 것이다.

현재 프로세스와 같은 tgid를 가지는 thread들을 탐색한다. 종료되어 zombie상태가 된 thread를 찾는다. 헤당 thread가 사용하던 ustack주소를 사용자에게 전달한다.

zombie thread를 발견하면 stack주소를 전달하고 freeproc시킨다. 회수한 thread의 pid를 반환한다.

만약 종료된 thread가 없다면 sleep 시키고, thread가 종료될 때 wakeup에 의해 다시 실행된다.

```
void drain_siblings(struct proc *p){
  struct proc *sibling;

  int live_siblings;

  acquire(&wait_lock);
  for (;;){
    live_siblings = 0;
    for (sibling = proc; sibling < &proc[NPROC]; sibling++){
      if (sibling == p) continue;

      acquire(&sibling->lock);
      if (sibling->tgid == p->tgid && sibling->state != UNUSED){
        sibling->killed = 1;
        if (sibling->state == SLEEPING){
```

```
                sibling->state = RUNNABLE;
            }
            if (sibling->state == ZOMBIE){
                freeproc(sibling);
            } else {
                live_siblings = 1;
            }
        }
        release(&sibling->lock);
    }
    if (!live_siblings){
        break;
    }
    sleep(p, &wait_lock);

  }

  release(&wait_lock);
}
```

헬퍼함수 drain_siblings 구현

Argument로 주어진 process p를 기준으로 tgid가 같은 sibling들의 killed신호를 1로 바꾸는 역할을 한다. 만약 그 sibling thread가 sleeping상태이면 runnable상태로 바꾸어 종료될 수 있도록 하고 zombie상태라면 freeproc시킨다. Sibling들이 안전하게 종료될 때까지 sleep(p, &wait_lock)으로 잠든다.

```
int
kkill(int pid)
{
  struct proc *p;
  int target_tgid = -1;

  for (p = proc; p < &proc[NPROC]; p++){
    acquire(&p->lock);
    if (p->pid == pid && p->state != UNUSED){
      target_tgid = p->tgid;
      release(&p->lock);
      break;
```

```
      }
      release(&p->lock);
    }
    if (target_tgid == -1){
      return -1;
    }

    for (p = proc; p < &proc[NPROC]; p++){
      acquire(&p->lock);
      if (p->tgid == target_tgid && p->state != UNUSED){
        p->killed = 1;
        if (p->state == SLEEPING){
          p->state = RUNNABLE;
        }
      }
      release(&p->lock);
    }
    return 0;
}
```

Kkill 구현

처음에 모든 process를 순회를 돌며 pid를 가지는 process를 찾고 p->tgid를 가져온다.

그 뒤, p->tgid와 같은 값을 가지는 process들을 모두 찾아 killed = 1로 설정하고 자고 있다면 runnable상태로 바꾸어 종료될 수 있게한다.

```
int

kwait(uint64 addr)
{
    struct proc *pp;
    int havekids, pid;
    struct proc *p = myproc();

    acquire(&wait_lock);

    for(;;){
      // Scan through table looking for exited children.
      havekids = 0;
      for(pp = proc; pp < &proc[NPROC]; pp++){
```

```
        if (pp->parent != p)
            continue;

        //modified
        if (pp->group_leader != pp)
            continue;

        // make sure the child isn't still in exit() or swtch().
        acquire(&pp->lock);
```

kwait()구현

pp->group_leader만 wait 할 수 있도록, if 문을 추가하였다.

```
int
kexec(char *path, char **argv)
{
  char *s, *last;
  int i, off;
  uint64 argc, sz = 0, sp, ustack[MAXARG], stackbase;
  struct elfhdr elf;
  struct inode *ip;
  struct proghdr ph;
  pagetable_t pagetable = 0, oldpagetable;
  struct proc *p = myproc();

  if (p->group_leader != p)
    return -1;

  begin_op();

  // Open the executable file.
  if((ip = namei(path)) == 0){
    end_op();
    return -1;
  }
  ilock(ip);

  // Read the ELF header.
```

```
if(readi(ip, 0, (uint64)&elf, 0, sizeof(elf)) != sizeof(elf))
    goto bad;

// Is this really an ELF file?
if(elf.magic != ELF_MAGIC)
    goto bad;

if((pagetable = proc_pagetable(p)) == 0)
    goto bad;

// Load program into memory.
for(i=0, off=elf.phoff; i<elf.phnum; i++, off+=sizeof(ph)){
    if(readi(ip, 0, (uint64)&ph, off, sizeof(ph)) != sizeof(ph))
        goto bad;
    if(ph.type != ELF_PROG_LOAD)
        continue;
    if(ph.memsz < ph.filesz)
        goto bad;
    if(ph.vaddr + ph.memsz < ph.vaddr)
        goto bad;
    if(ph.vaddr % PGSIZE != 0)
        goto bad;
    uint64 sz1;
    if((sz1 = uvmalloc(pagetable, sz, ph.vaddr + ph.memsz, flags2perm(ph.flags))) == 0)
        goto bad;
    sz = sz1;
    if(loadseg(pagetable, ph.vaddr, ip, ph.off, ph.filesz) < 0)
        goto bad;
}
iunlockput(ip);
end_op();
ip = 0;

p = myproc();
uint64 oldsz = p->mm->sz;

// Allocate some pages at the next page boundary.
// Make the first inaccessible as a stack guard.
// Use the rest as the user stack.
sz = PGROUNDUP(sz);
```

```
uint64 sz1;
if((sz1 = uvmalloc(pagetable, sz, sz + (USERSTACK+1)*PGSIZE, PTE_W)) == 0)
    goto bad;
sz = sz1;
uvmclear(pagetable, sz-(USERSTACK+1)*PGSIZE);
sp = sz;
stackbase = sp - USERSTACK*PGSIZE;

// Copy argument strings into new stack, remember their
// addresses in ustack[].
for(argc = 0; argv[argc]; argc++) {
    if(argc >= MAXARG)
        goto bad;
    sp -= strlen(argv[argc]) + 1;
    sp -= sp % 16; // riscv sp must be 16-byte aligned
    if(sp < stackbase)
        goto bad;
    if(copyout(pagetable, sp, argv[argc], strlen(argv[argc]) + 1) < 0)
        goto bad;
    ustack[argc] = sp;
}
ustack[argc] = 0;

// push a copy of ustack[], the array of argv[] pointers.
sp -= (argc+1) * sizeof(uint64);
sp -= sp % 16;
if(sp < stackbase)
    goto bad;
if(copyout(pagetable, sp, (char *)ustack, (argc+1)*sizeof(uint64)) < 0)
    goto bad;

// a0 and a1 contain arguments to user main(argc, argv)
// argc is returned via the system call return
// value, which goes in a0.
p->trapframe->a1 = sp;

// Save program name for debugging.
for(last=s=path; *s; s++)
    if(*s == '/')
        last = s+1;
```

```
    safestrcpy(p->name, last, sizeof(p->name));

    drain_siblings(p);

    // Commit to the user image.
    acquire(&p->mm->lock);
    oldpagetable = p->mm->pagetable;
    p->mm->pagetable = pagetable;
    p->mm->sz = sz;
    release(&p->mm->lock);
    p->trapframe->epc = elf.entry;    // initial program counter = ulib.c:start()
    p->trapframe->sp = sp; // initial stack pointer
    proc_freepagetable(oldpagetable, oldsz);

    return argc; // this ends up in a0, the first argument to main(argc, argv)

  bad:
    if(pagetable)
        proc_freepagetable(pagetable, sz);
    if(ip){
        iunlockput(ip);
        end_op();
    }
    return -1;
}
```

Kexec 구현

```
if (p->group_leader != p)
        return -1;
```

를 추가하여, groupleader만이 아래 코드를 접근할 수 있게 해주었다.

헬퍼함수 drain_siblings를 사용하여, 현재 같은 tgid를 사용하는 다른 모든 thread들을 제거한다.

```
int
thread_create(void (*fn)(void *), void *arg, int n_pages)
{
    void *stack_bottom = malloc(n_pages * PGSIZE);
    if (stack_bottom == 0) return -1;
```

```
    void *stack_top = (void*)((uint64)stack_bottom + n_pages * PGSIZE);

    int pid = clone(fn, arg, stack_top, n_pages, CLONE_VM);

    if (pid < 0){
        free(stack_bottom);
        return -1;
    }

    return pid;
}

int
thread_join(void)
{
    void *stack    = 0;

    int pid = join(&stack);
    if (pid < 0) return -1;

    if (pid >= 0 && stack != 0){
        free(stack);
    }
    return pid;
}
```

User/thread.c의 일부분이다.

Thread_create에서

Stack_top은 하향식이므로 stack_bottom에서 page size * 개수 만큼 더해주었다.
clone systemcall로 thread를 만들고 그 pid를 return 한다.

Thread_join에서.

Join()시스템콜을 호출하여 종료된 thread를 기다린다.

커널의 kjoin()으로부터 종료된 thread의 stack주소를 전달받는다.

해당 thread가 사용하던 user stack메모리를 free로 해제한다.

종료된 thread의 pid를 반환한다.

2. Design

A. 메모리 및 가상 주소 공간 설계

프로세스 생성 시에는 mm_alloc()을 통해 독자적인 mm_struct를 할당 받고 가상 메모리를 복사하지만, thread생성 시에는 부모의 p->mm 포인터를 그대로 복사하고 참조계수만 mm_get()로 증가시킵니다. find_free_tf_va()함수를 설계하여 TrapFrame시작 주소부터 아래 방향으로 PGSIZE 만큼 격차를 두며 비어있는 가상 주소 슬롯을 탐색합니다.

B. 유저공간과 커널의 역할 분담

유저 영역(uthread.c): thread가 실행될 때 유저 스택 메모리를 malloc()으로 안전하게 할당한 뒤 clone() 시스템 콜에 stack_top 주소를 넘겨주며, thread가 조인되어 소멸할 때 free() 처리를 전담합니다.

커널 공간(proc.c): 커널은 전달받은 스택 포인터를 새 thread의 trapframe->sp에 이식하고 가상 주소 공간 매핑을 유지관리합니다.

3. implementation

Step1.
mm_get(), mm_put()구현:

mm_get은 안전하게 락을 획득한 후 참조계수를 1 늘려 현재 주소 공간을 참조하는 thread가 추가되었음을 기록합니다.

mm_put() 은 참조 계수를 1 줄이고, 계수가 0이 되는 순간 더 이상 참조하는 thread가 없다고 판단하여 가상공간 매핑을 해제하고 페이지 테이블 메모리 및 구조체를 완전히 커널에 반환합니다.

find_tf_ta_va() 구현:

0부터 nthread-1 까지 순회하며 해당 가상 주소를 점유한 프로세스가 있는지 테이블을 모두 조사하여 빈 슬롯이 있다면 주소를 반환합니다.

freeproc() 수정:

thread가 완전히 해제될 때 공유 페이지 테이블에서 해당 thread가 할당받아 쓰던 고유

트랩프레임 가상 주소 영역만 uvmunmap하도록 수정하여 페이지 누수를 방지했습니다.

Step2

kclone():

새 프로세스 제어 블록 (np)에 부모의 p->mm포인터를 넘겨주고 mm_get()을 호풀합니다.

find_free_tf_va()를 호출하여 가상 주소를 선점한 뒤, mappages를 통해 새 thread의 물리 트렙프레임 페이지를 해당 가상 주소에 매핑합니다.

Trapframe->sp를 유저 스택주소로 설정하고, epc를 실행함수 포인터로, a0를 함수인자로 바인딩합니다.

kjoin():

현재 프로세스와 동일한 thread group id를 가진 스레드들 중 ZOMBIE상태인 thread를 탐색합니다. 좀비 thread를 발견하면 copyout()을 통해 해당 thread의 ustack을 return하여 유저 라이브러리가 메모리를 해제할 수 있도록 돕고, freeproc()으로 커널자원을 가져옵니다.

user/uthread.c:

thread_create에서 n_pages * PGSIZE만큼 유저 메모리를 할당하고 하향식 stack특성에 맞게 상단 주소를 stact_top을 clone()시스템콜로 전달합니다. thread_join은 자원이 반환되면 자동으로 유저스택을 free()처리합니다.

Step3.

Drain_siblings() 헬퍼함수 구현:

Thread그룹리더가 종료되거나 exec를 실행할 때, 자신을 제외한 그룹내의 모든 형제 thread들을 찾아 killed = 1 플래그를 세우고 state가 SLEEPING인 thread를 깨웁니다. 이후 모든 형제 thread가 ZOMBIE가 되어 안전하게 freeproc으로 회수될때 까지 wait sleep합니다.

kexec(), kexit(), kwait(), kkill() 수정:

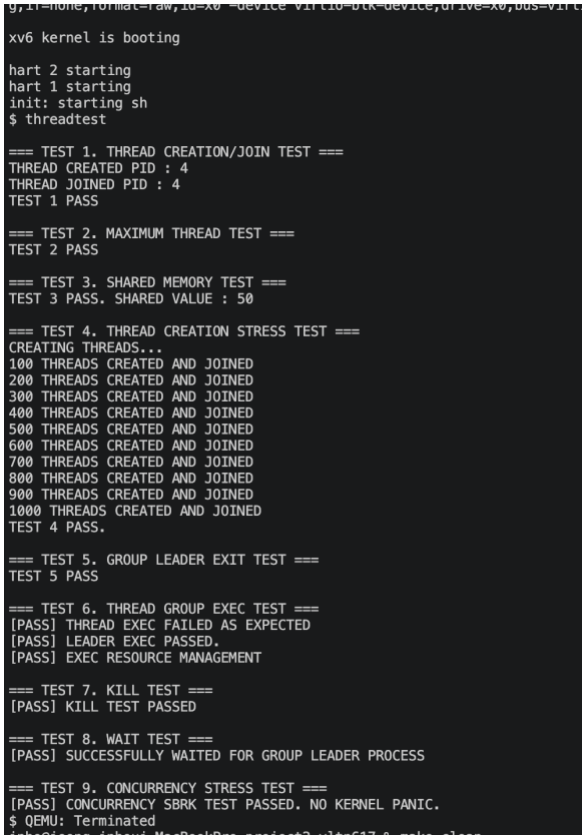
kexit(): 리더가 호출한 경우 drain_siblings()를 사용하여 전 수하 thread들을 동기적으로 정리하고 최종 종료됩니다.

kexec: 리더가 아닌 thread 호출을 차단하고, draing_siblings를 통해 기존 형제 thread들을 정리합니다.

kwait: 기존 부모-자식 수확과정에서 일반 thread들은 무시하고 thread group leader만 자식으로 인정하여 수확하도록 제한합니다.

kkill: 특정 pid에 시그널이 도달하면 해당 thread가 속한 전체 tgid를 추적하여 그룹내 모든 실행 유닛의 killed 플래그를 활성화합니다.

4. Results



5. Troubleshooting

락순서가 꼬여서 CPUS=1에서 작동하지만, 멀티 코어환경에서 작동하지 않는 상황이 발생하였다. 락 순서를 단일화 하기 위해, 코드를 재배치 했다.